BSE3211 – Software Verification and Testing

Unit Testing Exercise: Report

Group: GROUP L

Name	Student Number	Registration Number
Musheija Abraham	2300712139	23/U/12139/EVE
Lukaanya George	2300700696	23/U/0696
Kizito Ryan	2300710160	23/U/10160/EVE
Okema Paul Mark	2300716648	23/U/16648/EVE
Mwesigwa Sean Duncan	2100712248	21/U/12248/PS

---

1. Introduction

This report documents the unit testing exercise for the **Meeting Planner** system, a Java application that manages calendars for employees and rooms in an organisation. The system allows users to book meetings, block off vacation time, check availability, and print agendas.

The exercise required:

1. Formulating an informal test plan covering normal and erroneous inputs.
2. Implementing those tests in the **JUnit 4** framework.
3. Running the tests and analysing the results to identify faults in the system.

The source code was provided as a Maven project (`meetingplanner`) and contains a number of intentionally seeded bugs that the tests were designed to expose.

---

2. System Under Test

2.1 High-Level Features

The Meeting Planner supports six high-level functions:

#	Feature
1	Booking a meeting
2	Booking vacation time (all-day block)
3	Checking availability for a room
4	Checking availability for a person
5	Printing the agenda for a room
6	Printing the agenda for a person

2.2 Key Classes

Class	Responsibility
`Meeting`	Entity holding month, day, start hour, end hour, attendees, room, and description
`Calendar`	Manages a 3-D ArrayList of meetings indexed by month → day → meeting. Core logic: `addMeeting`, `isBusy`, `checkTimes`, `printAgenda`, `getMeeting`, `removeMeeting`
`Person`	Holds a name and a `Calendar`; delegates all scheduling operations to it
`Room`	Holds a room ID and a `Calendar`; delegates all scheduling operations to it
`Organization`	Holds 5 pre-configured employees and 5 rooms
`TimeConflictException`	Checked exception thrown for invalid dates, times, or booking conflicts

2.3 Internal Calendar Design

`Calendar` stores meetings in a three-dimensional `ArrayList` indexed as `[month][day][meetingIndex]`, where months are 1–12 and days are 1–31. To block off days that do not exist (e.g., February 29), the constructor pre-populates those slots with placeholder `Meeting` objects whose description is `"Day does not exist"`. The `addMeeting` method skips these placeholders when checking for conflicts, while `isBusy` does not — meaning `isBusy` correctly detects those slots as always occupied.

---

3. Test Plan Summary

The test plan was organised into nine sections, one per class or functional area. For each section, test cases were designed to cover three categories:

- **Normal inputs** – expected use of the feature.
- **Boundary values** – edge cases such as hour 0, hour 23, day 1, day 31, month 1, month 12.
- **Illegal inputs** – values outside valid ranges, double-booking attempts, and non-existent dates.

A total of **34 named test cases** were identified, mapped to test IDs (C01–C34, M01–M06, P01–P06, R01–R04, O01–O06). Particular attention was given to the five seeded bugs documented in the lab guide, each of which was assigned at least one dedicated test.

---

4. Test Implementation

Tests were written in **JUnit 4** across five test classes, one per production class under test.

4.1 Test Files and Method Counts

Test Class	Tests Written	Description
`CalendarTest`	36	Core calendar logic: add, busy check, validation, agenda, get/remove
`MeetingTest`	9	All constructors, getters/setters, `toString`, `addAttendee`
`PersonTest`	11	Meeting booking, conflict detection with name in message, agenda
`RoomTest`	12	Meeting booking, conflict detection with ID in message, agenda
`OrganizationTest`	8	Room and employee lookup (valid and invalid names/IDs)
Total	**76**	

4.2 Test Techniques Used

- `@Test(expected = ExceptionType.class)` — for tests that must throw a specific checked exception.
- `@Before` — to create a fresh `Calendar`, `Person`, or `Room` instance before each test, preventing state leakage between tests.
- `assertTrue` **/** `assertFalse` — to verify boolean states (busy/free).
- `assertEquals` — to verify field values on retrieved `Meeting` objects.
- `fail(message)` — to give an informative failure message when an unexpected exception is caught or when no exception is thrown where one is expected.
- `try/catch` **with** `instanceof` — for Bug 1, where `getMeeting`/`removeMeeting` do not declare `throws TimeConflictException`, making it impossible to catch it as a checked exception directly in a `catch` block.
- **Helper method** `timedMeeting()` — a private static helper in `CalendarTest` to create a `Meeting(month, day, start, end)` with a description set, working around the null-description NPE bug while keeping test code readable.

---

5. Test Execution Results

Tests were run using `mvn test` against the unmodified (buggy) production code.

5.1 Overall Results

Test Class	Run	Pass	Fail
`RoomTest`	12	12	0
`PersonTest`	11	11	0
`OrganizationTest`	8	8	0
`MeetingTest`	9	9	0
`CalendarTest`	36	27	9

Test Class	Run	Pass	Fail
Total	**76**	**67**	**9**

All 9 failures are **intentional** — they are bug-exposing tests that document a defect in the production code. No test failed for an unintended reason.

5.2 Failing Tests (Bug-Exposing)

Test Method	Bug Exposed	Failure Message
`testAddMeeting_december_shouldBeValid`	Bug 4	`December (month 12) is a valid month but was rejected — Month does not exist.`
`testCheckTimes_december_shouldBeValid`	Bug 4	`checkTimes should accept December (month 12) — Month does not exist.`
`testAddMeeting_sameStartEndHour_shouldBeValid`	Bug 5	`A meeting starting and ending at the same hour should be valid — Meeting starts before it ends.`
`testCheckTimes_sameStartEnd_shouldBeValid`	Bug 5	`checkTimes should allow start == end — Meeting starts before it ends.`
`testIsBusy_november30_shouldBeFree`	Bug 3	`November 30 is valid and should be free on a fresh calendar`
`testGetMeeting_invalidMonth_shouldThrowTimeConflictException`	Bug 1	`Expected TimeConflictException but got IndexOutOfBoundsException: Index -1 out of bounds for length 14`
`testRemoveMeeting_invalidMonth_shouldThrowTimeConflictException`	Bug 1	`Expected TimeConflictException but got IndexOutOfBoundsException: Index -1 out of bounds for length 14`
`testAddMeeting_nullDescription_secondAddThrowsNPE`	Unlisted	`Bug: addMeeting NPEs when an existing meeting has a null description`
`testAddMeeting_november31_shouldThrow`	Unlisted	`Bug: November 31 does not exist but addMeeting accepted it without error`

---

6. Bugs Found

6.1 Seeded Bugs (from Lab Guide)

Bug 1 — `getMeeting` and `removeMeeting` have no date/time validation

Location: `Calendar.java` — `getMeeting(int month, int day, int index)` and `removeMeeting(int month, int day, int index)`

Description: Neither method calls `checkTimes()` before accessing the internal array. Passing an invalid month (e.g., `-1`) causes `ArrayList.get(-1)` to throw `IndexOutOfBoundsException` instead of the expected `TimeConflictException`. This breaks the contract of the public API.

Evidence: JUnit reports `Expected TimeConflictException but got IndexOutOfBoundsException: Index -1 out of bounds for length 14`.

Fix: Both methods should call `checkTimes(month, day, 0, 1)` (or an equivalent date-only validator) before accessing the array.

Bug 2 — Calendar initialises 14 months (indices 0–13)

Location: `Calendar.java` constructor

Description: The loop `for(int i = 0; i <= 13; i++)` initialises slots for months 0 through 13, giving the internal structure 14 month slots when only 12 are valid (1–12). Months 0 and 13 are accessible via `getMeeting` and `removeMeeting` (which bypass `checkTimes`). Although `checkTimes` correctly rejects months 0 and 13 through the normal API, the structural redundancy can cause unexpected behaviour when validation is bypassed.

Fix: Change `i <= 13` to `i <= 12` so the array only allocates slots 0–12 (using slot 0 as unused padding).

Bug 3 — November 30 is incorrectly blocked as a non-existent day

Location: `Calendar.java` constructor

Description: The constructor intends to block only November 31 (which does not exist) but instead also blocks November 30:

```
// Buggy code
occupied.get(11).get(30).add(new Meeting(11, 31, "Day does not exist"));
occupied.get(11).get(31).add(new Meeting(11, 31, "Day does not exist"));
```

Slot `get(30)` corresponds to **day 30**, which is a valid date. As a result, `isBusy(11, 30, ...)` always returns `true` on a fresh calendar, making it impossible to book anything on November 30.

Evidence: `assertFalse(calendar.isBusy(11, 30, 9, 10))` fails — the calendar reports November 30 as perpetually occupied.

Fix: Remove the first line (`occupied.get(11).get(30).add(...)`) so that only slot 31 is blocked.

Bug 4 — December (month 12) is rejected by `checkTimes`

Location: `Calendar.java` — `checkTimes(int mMonth, int mDay, int mStart, int mEnd)`

Description: The month validation uses a `>=` comparison instead of `>`:

```
// Buggy
if(mMonth < 1 || mMonth >= 12){
    throw new TimeConflictException("Month does not exist.");
```

```
}
```

Because `12 >= 12` is true, December is treated as an invalid month. Any call to `addMeeting`, `isBusy`, `getMeeting`, or `removeMeeting` in December throws `TimeConflictException("Month does not exist.")`.

Evidence: `calendar.addMeeting(new Meeting(12, 1, 9, 10))` throws `TimeConflictException: Month does not exist.`

Fix: Change `mMonth >= 12` to `mMonth > 12`.

Bug 5 — Meetings that start and end in the same hour are rejected

Location: `Calendar.java` — `checkTimes`

Description: The time ordering check uses `>=` instead of `>`:

```
// Buggy
if(mStart >= mEnd){
    throw new TimeConflictException("Meeting starts before it ends.");
}
```

This means `start == end` (e.g., a one-hour block from 9 to 9) throws an exception, when the intent is only to reject meetings where start is strictly after end.

Evidence: `Calendar.checkTimes(3, 15, 9, 9)` throws `TimeConflictException: Meeting starts before it ends.`

Fix: Change `mStart >= mEnd` to `mStart > mEnd`.

6.2 Additional Bugs Discovered During Testing

Bug 6 — `addMeeting` NPEs when an existing meeting has a null description

Location: `Calendar.java` — `addMeeting`

Description: The conflict-detection loop calls `toCheck.getDescription().equals("Day does not exist")` without a null guard. The `Meeting(int month, int day, int start, int end)` constructor does not initialise `description`, leaving it `null`. Consequently, adding a second meeting to any day that already contains a description-less meeting causes `NullPointerException`.

```
// Buggy
if(!toCheck.getDescription().equals("Day does not exist")){
```

Evidence: `testAddMeeting_nullDescription_secondAddThrowsNPE` — adding two consecutive meetings to the same day without descriptions throws `NullPointerException: Cannot invoke "String.equals(Object)" because the return value of "Meeting.getDescription()" is null`.

Fix: Use the null-safe form: `if(!"Day does not exist".equals(toCheck.getDescription()))`.

Bug 7 — `addMeeting` silently accepts non-existent days

Location: `Calendar.java` — `addMeeting`

Description: The "Day does not exist" conflict check inside `addMeeting` skips placeholder meetings using a description equality check. This means adding a meeting to a non-existent date (e.g., November 31) is accepted without error because the placeholder meeting is skipped. The booking silently succeeds even though the date

is invalid.

Evidence: `testAddMeeting_november31_shouldThrow` fails with: `Bug: November 31 does not exist but addMeeting accepted it without error.`

Fix: `addMeeting` should validate whether the day is blocked by a "Day does not exist" placeholder *before* allowing a booking, not simply skip those meetings in the conflict loop.

Bug 8 — `addAttendee` *throws NPE on Meetings constructed without attendees*

Location: `Meeting.java` — `addAttendee(Person attendee)`

Description: The `attendees` field is an `ArrayList<Person>` that is only initialised by the full 7-argument constructor. The default constructor and the `Meeting(month, day, start, end)` constructor leave `attendees` as `null`. Calling `addAttendee()` on such a meeting immediately throws `NullPointerException`.

Evidence: `testAddAttendee_defaultConstructor_throwsNPE` in `MeetingTest` catches the NPE and records it as a known bug.

Fix: Initialise `attendees = new ArrayList<>()` in all constructors that do not accept an attendees parameter.

Bug 9 — `Meeting.toString()` *and* `printAgenda` *NPE when* `room` *is null*

Location: `Meeting.java` — `toString()`

Description: `toString()` unconditionally calls `room.getID()` and iterates over `attendees`. Any meeting not created via the full 7-argument constructor has `room = null`, causing `NullPointerException`. Since `Calendar.printAgenda` calls `toString()` on every meeting it encounters (including "Day does not exist" placeholders, which have `room = null`), calling `printAgenda` on months that contain placeholder meetings (February, April, June, September, November) will also NPE.

Fix: Add null-safety checks in `toString()`, or ensure all Meeting constructors initialise `room` and `attendees` to safe defaults.

---

7. Summary of All Bugs

#	Type	Location	Description	Severity
1	Seeded	`Calendar.getMeeting`, `Calendar.removeMeeting`	No date validation; wrong exception type thrown	Medium
2	Seeded	`Calendar` constructor	14-month array (indices 0–13) instead of 12	Low
3	Seeded	`Calendar` constructor	November 30 incorrectly blocked as non-existent	High
4	Seeded	`Calendar.checkTimes`	`>=12` rejects December	High

#	Type	Location	Description	Severity
5	Seeded	`Calendar.checkTimes`	`>=` rejects same-hour start/end	Medium
6	Discovered	`Calendar.addMeeting`	`getDescription()` called without null guard → NPE	High
7	Discovered	`Calendar.addMeeting`	Non-existent days (e.g., Nov 31) silently accepted	High
8	Discovered	`Meeting.addAttendee`	Attendees list not initialised in most constructors → NPE	Medium
9	Discovered	`Meeting.toString`, `Calendar.printAgenda`	`room` null in most constructors → NPE in toString/printAgenda	Medium

---

8. Conclusion

A total of **76 JUnit unit tests** were written across five test classes, achieving broad coverage of the Meeting Planner's public API. Of those, **67 tests pass** on the current (buggy) code, confirming that the core structure and many features behave correctly. The remaining **9 tests fail intentionally**, each producing a clear failure message that pinpoints a specific defect.

The five bugs seeded by the lab authors were all successfully detected: Bugs 3 and 4 are the most severe (blocking valid dates entirely), while Bug 5 prevents any same-hour meetings. Beyond the seeded faults, four additional bugs were discovered through exploratory testing: a null description NPE in the conflict check, silent acceptance of non-existent dates, uninitialized attendees lists, and NPE-prone `toString` calls.

Fixing all nine bugs would result in all 76 tests passing without modification.